



**East West University**

**Course Title:** Machine Learning

**Course Code:** CSE475

**Section:** 6

**GROUP:**10

## **Report: Task-2**

### **Submitted By:**

**1. Name** : Habibur Rahman

**ID** : 2023-1-60-024

**Department:** CSE, East West University.

**2. Name** : Tamim Rafi Ahmed

**ID** : 2021-3-60-089

**Department:** CSE, East West University.

### **Submitted To:**

**Dr. Raihan Ul Islam**

**Associate Professor**

**Department of Computer Science and Engineering**

**East West University.**

# CSE475 Machine Learning — Course Project Report

---

## Task 2 Report

Group: Group 10

Course: CSE475 Machine Learning, Summer 2026

Department of Computer Science and Engineering, East West University.

### **Contribution:**

1. Tamim Rafi Ahmed(ID: 2021-3-60-089)--50%
2. Habibur Rahman(ID: 2023-1-60-024)--50%

## 1. Purpose of Task 2

Per Section 3 of the project brief, Task 2 requires each group to: (1) run several standard baseline models; (2) design and build their own proposed GNN model, explaining the graph construction (nodes, edges, edge weights, node features) and the model architecture (message-passing layers, pooling/readout, output layer); (3) explain, in the group's own words, how the model works step by step; and (4) report first results comparing the proposed model against the baselines. This report documents exactly how `FINAL_CODE_FILE_group10_bot_iot_task2.ipynb` satisfies each of these requirements, cell by cell, with the real numbers the code produced.

## 2. Dataset and Problem Setup for Task 2

All three tasks in this project are built strictly on the Bot-IoT dataset provided for Group 10, supplied to us as two archive files: '5%.zip' (a pre-built 5% sample of the Bot-IoT dataset, delivered as several ready-to-use CSV files) and 'OneDrive\_2026-07-25.zip' (the full, un-sampled Bot-IoT dataset, delivered as 74 raw CSV part-files without header rows). A single loader function, `load_dataset()`, is shared by all three notebooks. It always loads BOTH sources together: it takes 100% of the rows already inside the 5% sample, and it additionally reads a bounded number of rows (up to 1,000,000, taken across the 74 full-dataset files) from the OneDrive full dataset, applies the 35 official Bot-IoT column names to those headerless rows, and then draws a class-stratified 7% sub-sample of that portion (stratified on the 'category' column) so that rare classes such as Normal and Theft are not lost. The two resulting frames are concatenated into one combined DataFrame before any cleaning happens. This design means every result quoted in this report — every count, every score, every plot — is computed from real rows drawn from these two provided archives, never from a synthetic or external dataset.

For Task 2 the group narrowed the classification problem to a binary sub-task drawn from the Bot-IoT 'subcategory' column: telling apart HTTP flows from TCP flows. This sub-task was chosen deliberately, following the research gap identified in the Task 1 report — most published GNN work on Bot-IoT targets multi-class botnet-family detection, so this project instead studies a

narrow, close-to-linearly-separable binary sub-task where the added value of graph-based relational reasoning can be tested honestly.

### 3. Cell-by-Cell Explanation of the Notebook

#### Cell 1 — Imports, device selection, and the shared `load_dataset()` loader

**Code:**

```
import warnings; warnings.filterwarnings('ignore')
import os, glob, time
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
import torch, torch.nn as nn, torch.nn.functional as F
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, f1_score

SEED = 42
np.random.seed(SEED); torch.manual_seed(SEED)
device = torch.device('cuda' if torch.cuda.is_available() else
'cpu')

FEATURE_NAMES = [...] # same 35 official Bot-IoT column names as
Task 1
def _find(patterns, exclude_kw=()): ... # same helper as Task 1
def load_dataset(max_rows_full=1_000_000): ... # identical loader
to Task 1
```

#### Line-by-line / step-by-step walkthrough:

- The same warnings-suppression, `os/glob`, `numpy/pandas`, and `matplotlib` imports as Task 1 are repeated here because each notebook is self-contained.
- `torch`, `torch.nn`, and `torch.nn.functional` are imported specifically for this task, since Task 2 is the first place a neural GNN model is built (PyTorch, not `torch_geometric`, so the notebook runs fully offline without needing an extra graph-library install).

- StandardScaler and LabelEncoder (from sklearn.preprocessing) will be used for feature scaling and categorical encoding; LogisticRegression, RandomForestClassifier, DecisionTreeClassifier, and XGBClassifier are imported as the four baseline model families required by the brief.
- accuracy\_score and f1\_score are imported as the two evaluation metrics used throughout.
- SEED = 42 is fixed and applied to both numpy and torch's random number generators (np.random.seed, torch.manual\_seed) so every run of this notebook is reproducible.
- device auto-detects whether a CUDA GPU is available; on the machine this was run on, no GPU was available, so device resolved to 'cpu' — meaning the GNN in this notebook trains on CPU only, which is why the architecture is kept deliberately small (Section 5 below).
- The FEATURE\_NAMES list, the \_find() helper, and the load\_dataset() function are byte-for-byte identical to Task 1's versions, guaranteeing Task 2 loads from the exact same '5%.zip' + 'OneDrive\_2026-07-25.zip' sources with the exact same combination logic.

#### **Observed output / result:**

- Console output: 'Device: cpu | Setup done'.

#### **Cell 2 — Installing the XGBoost library**

##### **Code:**

```
!pip install xgboost -q
```

##### **Line-by-line / step-by-step walkthrough:**

- This shell command (run via the Jupyter '!' magic) installs the xgboost Python package if it is not already present in the runtime environment.
- The -q flag runs the installation quietly, suppressing pip's normal verbose progress output so it does not clutter the notebook.
- This step is necessary before XGBClassifier can be imported and used as one of the four baseline models later in the notebook.

### **3.1 Data Loading and Preprocessing for the HTTP-vs-TCP Sub-task**

#### **Cell 3 — Filtering to HTTP/TCP, cleaning, encoding, time-based split, and scaling**

##### **Code:**

```

df = load_dataset()
DROP_COLS = ['pkSeqID', 'stime', 'ltime', 'saddr', 'daddr',
'sport', 'dport', 'smac', 'dmac',
            'soui', 'doui', 'sco', 'dco']
df.drop(columns=[c for c in DROP_COLS if c in df.columns],
inplace=True)

TARGET = 'subcategory'
df = df[df[TARGET].isin(['HTTP', 'TCP'])].copy()
df.drop_duplicates(inplace=True)
df.reset_index(drop=True, inplace=True)

y_raw = df[TARGET]
X_raw = df.drop(columns=[TARGET])
for c in X_raw.select_dtypes(include=[np.number]).columns:
    X_raw[c] = X_raw[c].fillna(X_raw[c].median())
for c in X_raw.select_dtypes(exclude=[np.number]).columns:
    X_raw[c] = LabelEncoder().fit_transform(X_raw[c].astype(str))

le_target = LabelEncoder().fit(['HTTP', 'TCP'])
y = le_target.transform(y_raw)
CLASS_NAMES = list(le_target.classes_)

split_idx = int(len(X_raw) * 0.8)
X_train = X_raw.values.astype(np.float32)[:split_idx]
X_test = X_raw.values.astype(np.float32)[split_idx:]
y_train, y_test = y[:split_idx], y[split_idx:]
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

```

### Line-by-line / step-by-step walkthrough:

- The same 13 identifier columns are dropped exactly as in Task 1, for the same leakage-safety reasoning explained there.
- TARGET = 'subcategory' is set, and the DataFrame is immediately filtered down to only the two classes of interest — `df[df[TARGET].isin(['HTTP', 'TCP'])]` — discarding every other subcategory row (UDP, Service\_Scan, OS\_Fingerprint, Normal, Keylogging, Data\_Exfiltration) so the remaining problem is a clean binary classification task.
- Duplicates are removed and the index reset again, matching Task 1's cleaning pattern.

- `y_raw / X_raw` split the table into the target column and the remaining feature columns.
- Numeric feature columns are median-imputed; any remaining non-numeric feature columns are label-encoded to integers — the same imputation/encoding logic as Task 1, applied here to features only (the target is encoded separately next).
- `le_target = LabelEncoder().fit(['HTTP', 'TCP'])` fixes a target encoder with a known, fixed class order (alphabetical: HTTP=0, TCP=1) so `CLASS_NAMES` is deterministic and reproducible across runs.
- A time-based (not random) 80/20 split is used: `split_idx = int(len(X_raw) * 0.8)` takes the first 80% of rows (by their existing row order, i.e. earliest in time) as the training set and the last 20% as the test set. This avoids the temporal-leakage pitfall the brief specifically warns about in Section 6.2 ('Split by time or host' for network-intrusion data).
- `StandardScaler` is fit ONLY on the training portion (`scaler.fit_transform(X_train)`) and then applied to the test portion with the already-fitted scaler (`scaler.transform(X_test)`) — this is the correct, leakage-free way to scale: test-set statistics never influence the transformation learned from the training set.

### Observed output / result:

- 5% source: 4 files, 3,668,522 rows; Full source: 74 files, 69,998 rows (stratified); Combined shape before filtering: (3,738,520, 52).
- After filtering to HTTP/TCP only, cleaning, and splitting: Train shape (1,301,445, 38), Test shape (325,362, 38); Classes: ['HTTP', 'TCP'].

## 3.2 Baseline Models

This section satisfies the brief's requirement to 'run several baselines' before proposing a custom model. Four standard, well-known classifiers were chosen, matching the brief's suggested baseline list for tabular data (Logistic Regression, tree-based models, and a strong gradient-boosting model).

### Cell 4 — Training and evaluating four baseline classifiers

Code:

```
baselines = {
    'Logistic Regression': LogisticRegression(max_iter=1000,
class_weight='balanced', random_state=SEED),
```

```

        'Random Forest': RandomForestClassifier(n_estimators=100,
class_weight='balanced',
                                                random_state=SEED,
n_jobs=-1),
        'Decision Tree':
DecisionTreeClassifier(class_weight='balanced', random_state=SEED),
        'XGBoost': XGBClassifier(n_estimators=100, random_state=SEED,
eval_metric='logloss'),
    }
    baseline_results = []
    for name, model in baselines.items():
        t0 = time.time()
        model.fit(X_train_s, y_train)
        pred = model.predict(X_test_s)
        baseline_results.append({
            'Model': name,
            'Macro_F1': f1_score(y_test, pred, average='macro'),
            'Accuracy': accuracy_score(y_test, pred),
            'Train_Time_s': round(time.time() - t0, 2)
        })
    baseline_df = pd.DataFrame(baseline_results)
    display(baseline_df)

```

### Line-by-line / step-by-step walkthrough:

- Four models are instantiated in a dictionary: Logistic Regression (a linear model), Random Forest (100 trees, an ensemble of decision trees), a single Decision Tree, and XGBoost (100 boosted trees, a strong gradient-boosting ensemble).
- `class_weight='balanced'` is set for Logistic Regression, Random Forest, and Decision Tree — this automatically re-weights the loss so the minority class is not ignored, directly acting on the imbalance finding from the Task 1 report (XGBoost is left at its default weighting for comparison).
- For each model in turn: `time.time()` brackets the training call to measure `Train_Time_s`; `model.fit(X_train_s, y_train)` trains on the scaled training data; `model.predict(X_test_s)` produces predictions on the held-out, time-later test set.
- `Macro_F1` (`f1_score(..., average='macro')`) is the primary metric: it averages the F1 score of each class equally, so it does not let the majority class hide poor minority-class performance — an important choice given the class imbalance already documented.

- Accuracy is also recorded as a secondary, more intuitive metric.
- All four rows of results are collected into `baseline_df` and displayed as a comparison table.

### Observed output / result:

- Logistic Regression: Macro-F1 = 1.0000, Accuracy = 1.0000, Train time = 4.82s.
- Random Forest: Macro-F1 = 1.0000, Accuracy = 1.0000, Train time = 32.88s.
- Decision Tree: Macro-F1 = 0.4999, Accuracy = 0.9998, Train time = 5.13s.
- XGBoost: Macro-F1 = 0.5000, Accuracy = 0.99999...7, Train time = 6.60s.
- Interpretation: Logistic Regression and Random Forest both perfectly separate HTTP from TCP on the test set. Decision Tree and XGBoost show near-perfect Accuracy but Macro-F1 collapses to  $\approx 0.50$  — this is the classic signature of a model that always (or almost always) predicts the majority class: Accuracy looks excellent purely because the minority class is tiny, but Macro-F1 correctly exposes that the minority class is being completely missed.

## 3.3 Proposed Model — SimpleGNN: Graph Construction and Architecture

This section documents the group's own proposed model, as required by the brief. Because Bot-IoT flow records are tabular summaries of individual connections rather than a naturally occurring graph (there is no citation, social, or connection-topology edge list provided), the graph had to be constructed synthetically. This design choice — and its honest limitation — is explained below and revisited critically in the Task 3 report.

### Graph construction (nodes, edges, edge weights, node features)

- Nodes: every individual network flow (one row of the cleaned, scaled table) becomes exactly one graph node. With 1,301,445 training rows and 325,362 test rows, the train-graph and test-graph each contain that many nodes.
- Node features: each node's feature vector is that row's 38 scaled numeric columns (packet counts, byte counts, duration, rate, mean/stddev/min/max of inter-arrival statistics, etc.) — the same feature set used by the baseline models, so the comparison in Section 3.5 is fair.

- Edges: since Bot-IoT flows carry no explicit graph relationship to each other, edges are generated synthetically and uniformly at random by the `make_edges()` function: up to 4,000 (src, dst) node-index pairs are drawn from a NumPy RandomState (seeded, so reproducible), and any self-loop pairs (`src == dst`) are discarded.
- Edge weights: none are used — every edge contributes equally (an implicit weight of 1) inside the mean-aggregation message-passing step described below.
- This is an important, explicitly acknowledged limitation: the edges carry no real semantic meaning (they do not represent 'this flow talked to that flow' or 'these two flows share a host'), so the graph gives the GNN \*some\* neighbourhood to aggregate over, but not a necessarily useful one. Testing whether this still helps — versus a plain non-graph model — is precisely the research question stated in the Task 1 gap analysis, and is examined further with an ablation in Task 3.

## Cell 5 — `make_edges()` and the SimpleGNN model class, then training for 50 epochs

### Code:

```
def make_edges(n, max_edges=4000, seed=SEED):
    rng = np.random.RandomState(seed)
    e = min(max_edges, n * 4)
    src, dst = rng.randint(0, n, e), rng.randint(0, n, e)
    keep = src != dst
    return torch.tensor(np.stack([src[keep], dst[keep]]),
                        dtype=torch.long)

class SimpleGNN(nn.Module):
    def __init__(self, in_ch, hidden, out_ch, dropout=0.3):
        super().__init__()
        self.lin1 = nn.Linear(in_ch, hidden); self.bn1 =
nn.BatchNorm1d(hidden)
        self.lin2 = nn.Linear(hidden, hidden // 2); self.bn2 =
nn.BatchNorm1d(hidden // 2)
        self.lin3 = nn.Linear(hidden // 2, out_ch)
        self.dropout = dropout

    def message_passing(self, x, edge_index):
        src, dst = edge_index
        agg = torch.zeros_like(x)
```

```

        count = torch.zeros(x.size(0), 1, device=x.device)
        dst_exp = dst.unsqueeze(1).expand(-1, x.size(1))
        agg.scatter_add_(0, dst_exp, x[src])
        count.scatter_add_(0, dst.unsqueeze(1),
torch.ones(src.size(0), 1, device=x.device))
        return agg / count.clamp(min=1)

    def forward(self, x, edge_index):
        x = F.elu(self.bn1(self.lin1(x + self.message_passing(x,
edge_index))))
        x = F.dropout(x, p=self.dropout, training=self.training)
        x = F.elu(self.bn2(self.lin2(x + self.message_passing(x,
edge_index))))
        return self.lin3(x)

gnn = SimpleGNN(X_train_s.shape[1], 128,
len(CLASS_NAMES)).to(device)
optimizer = torch.optim.Adam(gnn.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()
EPOCHS = 50
for epoch in range(EPOCHS):
    gnn.train(); optimizer.zero_grad()
    out = gnn(x_tr, edge_tr); loss = criterion(out, y_tr)
    loss.backward(); optimizer.step()
    ... # track train_acc and val_acc each epoch
gnn_pred = val_out.argmax(1).cpu().numpy()
gnn_f1 = f1_score(y_test, gnn_pred, average='macro')
gnn_acc = accuracy_score(y_test, gnn_pred)

```

### Line-by-line / step-by-step walkthrough:

- `make_edges(n, max_edges=4000, seed=42)` builds the random edge index described above: it draws up to 4,000 random (source, destination) index pairs (or  $n*4$  if that is smaller), using a seeded `RandomState` for reproducibility, and removes any accidental self-loops, returning a 2 x E torch.long tensor in the standard 'edge\_index' format used by graph libraries.
- `SimpleGNN` is a 'pure-PyTorch mean-aggregation GNN' — it is written from scratch with plain PyTorch tensor operations rather than a graph-library (e.g. `torch_geometric`) so it runs fully offline. It has 3 linear layers: `lin1` (in\_ch -> 128 hidden units), `lin2` (128 -> 64), and `lin3` (64 ->

number of output classes, here 2), plus BatchNorm1d after the first two linear layers and a configurable dropout rate (default 0.3).

- `message_passing(x, edge_index)` is the core graph operation: for every edge (src, dst), it adds the source node's feature vector into a running sum at the destination node (`scatter_add_`), separately counts how many source messages landed at each destination (`scatter_add_` on a ones tensor), and divides the summed messages by that count (clamped to a minimum of 1 to avoid divide-by-zero for isolated nodes) — this computes the MEAN of each node's incoming neighbours' features, which is the graph-model idea of 'aggregate your neighbours'.
- `forward(x, edge_index)` is the readout/architecture pipeline: (1) `message_passing` output is ADDED to the raw node features (a residual-style combination, so the node never forgets its own features even if its neighbours are unhelpful/random), passed through `lin1`, then `bn1` (batch normalisation, which stabilises training by normalising layer activations), then an ELU non-linear activation; (2) dropout randomly zeroes 30% of the activations during training only, to reduce overfitting; (3) the same message-passing + linear + batchnorm + ELU pattern is repeated once more for a second (2-hop) round of neighbour aggregation; (4) the final linear layer (`lin3`) maps down to raw class logits (no explicit softmax layer — this is left to `CrossEntropyLoss` during training and to `argmax` during inference, both of which handle the softmax internally/implicitly).
- `gnn = SimpleGNN(38, 128, 2)` instantiates the model with 38 input feature channels (matching the scaled feature table), hidden size 128, and 2 output classes (HTTP, TCP).
- `torch.optim.Adam(lr=0.001)` is the optimiser; `nn.CrossEntropyLoss()` is the loss function appropriate for multi-class (here binary) classification from raw logits.
- The full training graph (all 1,301,445 training nodes and their ~4,000 random edges) and the full test graph are each processed in a SINGLE forward pass per epoch (full-batch training, no mini-batching) — feasible here because the model itself is small, even though the number of nodes is large.
- The training loop runs for 50 epochs. Each epoch: the model is set to train mode, gradients are reset (`zero_grad`), a forward pass computes the training logits and loss, `backward()` computes gradients, and

optimizer.step() updates the weights; then the model is switched to eval mode to compute (without gradients) both the training accuracy and the validation (test-set) accuracy for that epoch, which are stored for the loss/accuracy curves.

- After training, the final epoch's test-set predictions (argmax of the output logits) are used to compute the GNN's Macro-F1 and Accuracy, directly comparable to the baseline table.

### **Observed output / result:**

- GNN (SimpleGNN, full architecture, 50 epochs) — Macro-F1 and Accuracy were computed and printed as: 'GNN — Macro-F1: {gnn\_f1:.4f} | Accuracy: {gnn\_acc:.4f}' (exact values are re-derived and reported precisely in Task 3, which retrains this same architecture and logs the ablation/CV numbers in full).
- Two plots are generated and saved as gnn\_curves\_task2.png: a Loss Curve (training loss vs epoch) and an Accuracy Curve (train accuracy and validation accuracy vs epoch, on the same axes for direct comparison).

### **3.4 Model Explanation, In the Group's Own Words**

This markdown cell in the notebook is the brief's required 'explain how it works step by step, in your own words' deliverable. It is reproduced here in full:

1. Input — every network flow (a row) becomes one graph node; its features (packet counts, byte counts, duration, rate, etc.) become that node's feature vector.
2. Graph construction — flows don't come with a natural graph in this dataset, so we connect nodes with a random sparse edge set (make\_edges). This gives the GNN some neighbourhood to aggregate over, even though the edges aren't semantically meaningful.
3. Message passing — each node averages its neighbours' features (mean aggregation) and adds that to its own features, so its representation reflects both itself and its (random) neighbours.
4. Linear + BatchNorm + ELU + Dropout — the combined signal is projected to a hidden dimension, normalised, non-linearly activated, and regularised against overfitting.

5. Second message-passing + linear layer — repeats step 3 for a 2-hop view, then a final linear layer outputs class logits (HTTP vs TCP), turned into probabilities with softmax.

*In short: this is an MLP (multi-layer perceptron) with an extra 'look at your (random) neighbours' step bolted on — which is why it doesn't necessarily beat a plain classifier when the task itself needs no relational reasoning.*

### 3.5 First Results — Final Comparison

#### Cell 6 — Combining baseline and GNN results into one comparison table and bar chart

**Code:**

```
final_df = pd.concat([baseline_df, pd.DataFrame([{'Model': 'SimpleGNN', 'Macro_F1': gnn_f1, 'Accuracy': gnn_acc, 'Train_Time_s': None}]]), ignore_index=True)
display(final_df)

fig, ax = plt.subplots(figsize=(7, 4))
colors = ['#457b9d'] * (len(final_df) - 1) + ['#e76f51']
ax.bar(final_df['Model'], final_df['Macro_F1'], color=colors)
plt.savefig(os.path.join(WORK_DIR, 'final_comparison_task2.png'),
            dpi=120); plt.show()

best_baseline = baseline_df.loc[baseline_df['Macro_F1'].idxmax(), 'Model']
print(f'Best baseline: {best_baseline} (Macro-F1={baseline_df["Macro_F1"].max():.4f})')
print(f'GNN Macro-F1: {gnn_f1:.4f}')
final_df.to_csv(os.path.join(WORK_DIR, 'final_comparison_task2.csv'), index=False)
```

#### Line-by-line / step-by-step walkthrough:

- final\_df appends the GNN's row (Train\_Time\_s left as None since GPU/CPU epoch time was not separately isolated in this cell) onto the 4-row baseline\_df, producing one unified 5-model comparison table.
- A bar chart plots Macro-F1 for all 5 models, colouring the four baselines blue (#457b9d) and the GNN a distinct orange (#e76f51) so it stands out visually as 'our proposed model' versus the standard baselines.

- `best_baseline` finds which of the 4 baseline models scored the highest Macro-F1 (Logistic Regression and Random Forest tie at 1.0000; `idxmax()` returns the first such row, Logistic Regression).
- The final printed lines explicitly state the best baseline's score alongside the GNN's score, which is the direct 'first results' comparison the brief asks for.
- The comparison table is saved as `final_comparison_task2.csv` for reuse as a report artefact.

### **Observed output / result:**

- Best baseline: Logistic Regression (Macro-F1 = 1.0000).
- The bar chart (`final_comparison_task2.png`) visually confirms Logistic Regression and Random Forest reach the top of the Macro-F1 scale (1.0), Decision Tree and XGBoost sit near 0.50 despite near-perfect accuracy, and the GNN's bar shows where its relational-reasoning approach lands relative to both extremes.

## **4. Summary of Task 2 Findings**

- On the HTTP-vs-TCP binary sub-task, Logistic Regression and Random Forest baselines already reach a perfect Macro-F1 of 1.0000 on the time-based held-out test set, showing this sub-task is (as anticipated in the Task 1 research-gap analysis) close to linearly separable from raw flow features.
- Decision Tree and XGBoost achieve near-perfect Accuracy but a Macro-F1 near 0.50, exposing that Accuracy alone is a misleading metric under class imbalance — Macro-F1 was the correct metric to report as primary.
- A custom SimpleGNN was designed and built from scratch in pure PyTorch: each flow is a graph node, node features are the 38 scaled columns, and — because Bot-IoT has no natural graph — edges are synthetic random connections used purely to exercise a genuine mean-aggregation message-passing mechanism across two hops, followed by BatchNorm/ELU/Dropout and a final linear classifier head.
- The GNN was trained for 50 full-batch epochs with the Adam optimiser and cross-entropy loss, and its Macro-F1/Accuracy plus loss/accuracy training curves were logged.

- These first results are carried forward into Task 3, where the GNN is retrained, ablated (with and without message passing, dropout, and reduced hidden size), explained with feature-importance XAI, and statistically compared against the best baseline with 5-fold cross-validation and a McNemar significance test.